浙江大学

本科实验报告

课程名称：　实验 5 常用组合电路模块的设计和应用

姓　　名：

学　　院：　信息与电子工程学院

系：

专　　业：　信息工程

学　　号：

指导教师：　屈民军，唐奕

2025 年 3 月 20 日

专业：信息工程
姓名：
学号：
日期：2025.3.19
地点：东四

浙江大学实验报告

课程名称：数字系统设计实验
指导老师：屈民军，唐奕
成绩：
实验名称：实验 5 常用组合电路模块的设计和应用

一、实验目的

1．掌握用 VerilogHDL 描述数据选择器、加法器和比较器等电路模块。

2．了解“自顶而下”的数字设计方法，掌握系统层次结构的设计。

3．掌握模块的调用方法，掌握参数的定义和参数传递的方法

4．掌握 ModelSim 功能仿真的工作流程，进一步了解 Vivado 的工作流程

5．认识到文件管理的重要性

二、实验设计原理与 Verilog 代码

2.1 任务一：设计两数之差的绝对值电路

1．任务要求：电路输入 aIn、bIn 为 4 位无符号二进制数，电路输出 out 为两数之差的绝对值，即 out=|aIn-bIn|。要求用多层次结构设计电路。

2．实验原理

（1）电路总体设计

两个无符号数之差的绝对值可以用

$$\mathrm{out} = \mathrm{Max}(\mathrm{aIn}, \mathrm{bIn}) - \mathrm{Min}(\mathrm{aIn}, \mathrm{bIn})$$

进一步化简为

$$\mathrm{out} = \mathrm{Max}(\mathrm{aIn}, \mathrm{bIn}) + (\sim\mathrm{Min}(\mathrm{aIn}, \mathrm{bIn}) + 1)$$

电路图如下图所示：

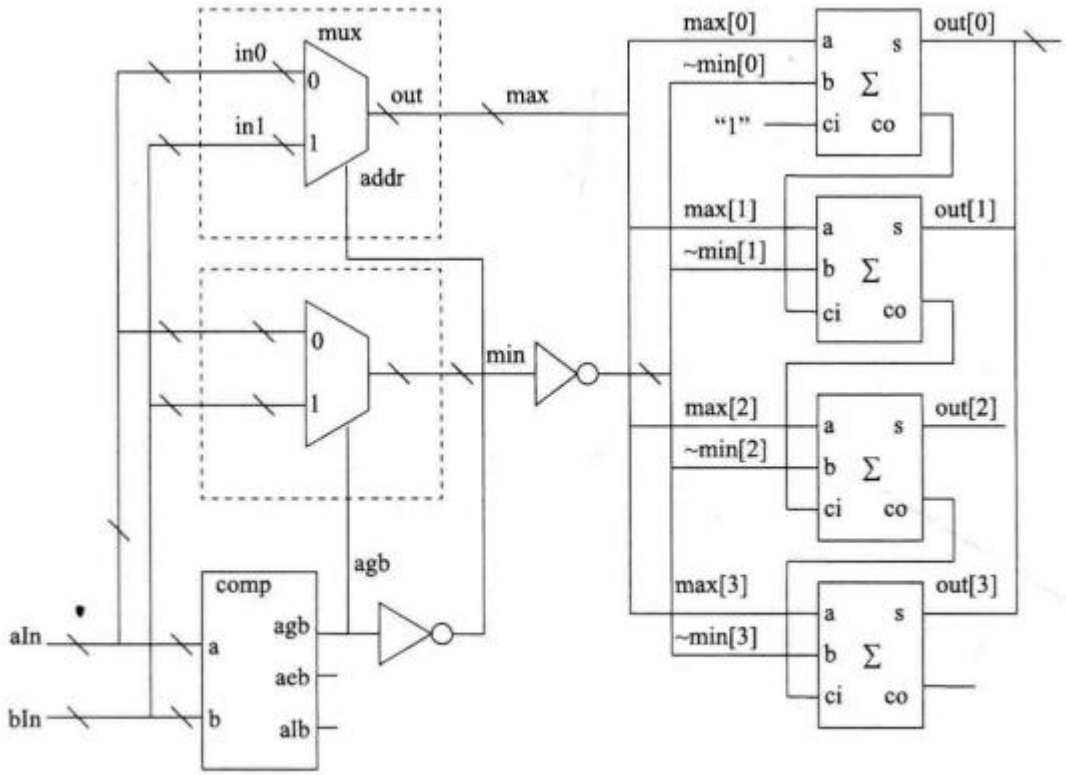


图 5.1　两数之差的绝对值电路的原理框图

主要由三部分组成：数值比较器（comp）实现比较两数的大小，agb 表示 a>b；比较结果 agb 控制两个数据选择器，分别选择输出两数的最大值 Max 和最小值 Min；最后将全加器级联形成的 4bit 串行加法器实现 Max 和 Min 的差，通过反码的方式实现

顶层模块的编写代码如下：

```
module abs_dif(aIn,bIn,out);
    input [3:0] aIn,bIn;
    output [3:0] out;

    //比较器示例
    wire agb;
    comp #(.n(4))comp_inst(.a(aIn), .b(bIn), .agb(agb), .aeb(), .alb());

    //数据选择器，括号前是模块端口，括号里是实际导线
    wire[3:0] max,min;
    mux_2to1 #(.n(4))mux1(.out(max), .in0(aIn), .in1(bIn), .addr(~agb));
    mux_2to1 #(.n(4))mux2(.out(min), .in0(aIn), .in1(bIn), .addr(agb));

    //全加器，4bit全加器的级联
    wire[2:0] c;
    full_adder adder(.a(max[0]), .b(~min[0]), .s(out[0]), .ci(1'b1), .co(c[0]));
    full_adder adder1(.a(max[1]), .b(~min[1]), .s(out[1]), .ci(c[0]), .co(c[1]));
    full_adder adder2(.a(max[2]), .b(~min[2]), .s(out[2]), .ci(c[1]), .co(c[2]));
    full_adder adder3(.a(max[3]), .b(~min[3]), .s(out[3]), .ci(c[2]), .co());
    endmodule
```

（2）电路模块设计

①数据选择器：实现 mux2to1，二选一数据选择器，功能如下：

表 5.2　数据选择器的功能表

addr	out[n-1:0]
0	in0[n-1:0]
1	in1[n-1:0]

为了适应顶层设计的调用要求，数据选择器的端口描述如下：

```
module  mux_2to1(out, in0,in1, addr);
parameter  n=1; //参数表示输入、输出数据的位数
output[n-1:0]  out;
input [n-1:0]   in0,in1;
input  addr;
```

实际代码如下：always 过程语句赋值，当括号内敏感信号发生变化时才执行功能，以实现选择，addr=1 时，out=in1，addr=0 时，out=in0

```
module mux_2to1(out, in0,in1, addr);

    parameter n=1;//输入输出数据位数

    output[n-1:0] out;
    input[n-1:0] in0,in1;
    input addr;
    reg[n-1:0] out;

    always @(in0 or in1 or addr)
        begin
            if(addr) out=in1;
            else out=in0;
        end
endmodule
```

②数值比较器：实现数值大小的比较，功能如下：

表 5.3　数据比较器的功能表

	agb	aeb	alb
a[n-1:0]>b[n-1:0]	1	0	0
a[n-1:0]=b[n-1:0]	0	1	0
a[n-1:0]<b[n-1:0]	0	0	1

为了适应顶层设计的调用要求，数值比较器的端口描述如下：

```
module comp(a,b,agb,aeb,alb);
    parameter n=1;//参数表示比较器的位数
    input [n-1:0] a, b;
    output  agb;   // a大于b
    output  aeb;   // a等于b
    output  alb;   // a小于b
```

实际代码如下：assign 实现连续赋值功能，用 bool 函数得到 a 和 b 的大小

```
module comp(a,b,agb,aeb,alb);
    parameter n=1;
    input [n-1:0] a,b;
    output agb;
    output aeb;
    output alb;
        assign agb=(a>b);
        assign aeb=(a==b);
        assign alb=(a<b);
    endmodule
```

③一位全加器：实现 1bit 的加法，有三个输入和两个输出，功能如下：

表 5.4　一位全加器的真值表

a	b	ci	s	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{cases} S_i = A_i \oplus B_i \oplus C_{i-1} \\ C_i = A_i B_i + B_i C_{i-1} + A_i C_{i-1} \end{cases} \tag{5.3}$$

为了适应顶层设计的调用要求，全加器的端口描述如下：

```
module  full_adder(a, b, s, ci, co);
    input  a, b, ci;
   output  s, co;
```

实际代码如下：assign 实现连续赋值功能，用逻辑函数进行赋值

```
module full_adder(a, b, s, ci, co);
    input a,b,ci;
    output s,co;

    assign s=a^b^ci;
    assign co=(a&b)|(b&ci)|(a&ci);

    endmodule
```

2.2 任务二：设计模式比较器（Mode-Dependent Comparator）电路

1．任务要求：电路输入为两个 8 位无符号二进制数 a、b 和一个模式控制信号 m；电路的输出为 8 位无符号二进制数 y。当 m=0 时，y=MAX（a，b）；而 m=1 时，则 y=MIN（a，b）。要求多层次结构设计电路

2．实验原理

（1）电路总体设计

实现的功能为根据 addr 的状态实现不同的功能，功能表如下

表 5.1　产生 addr 组合电路的功能表

m	agb	addr	备注
0	0	1	当 m=0，且 a≤b 时，y 取大值 b
0	1	0	当 m=0，且 a>b 时，y 取大值 a
1	0	0	当 m=1，且 a≤b 时，y 取小值 a
1	1	1	当 m=1，且 a>b 时，y 取小值 b

电路图如下：

主要由三部分组成：数值比较器（comp）实现比较两数的大小，agb 表示 a>b；比较结果 agb 和 Mode 模块实现同或后得到的 addr 控制数据选择器，选择输出两数的最大值或最小值（取决于 Mode 的状态）

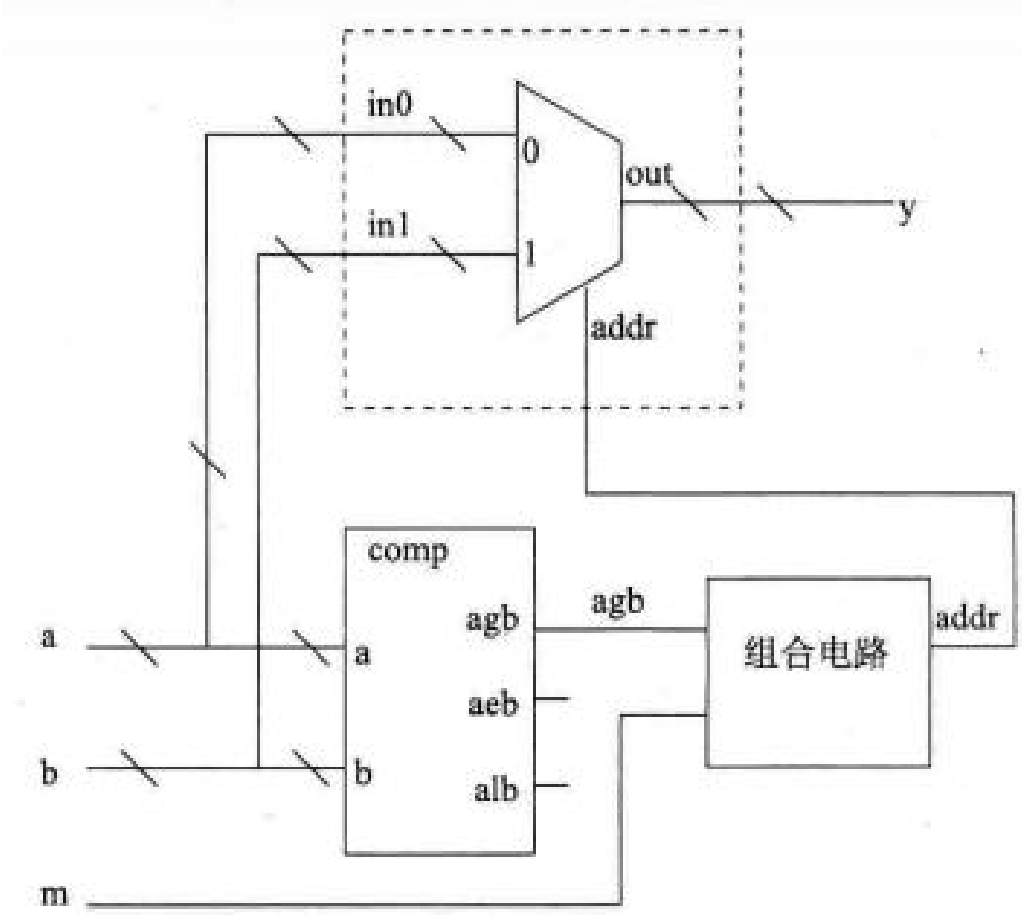


图 5.2　模式比较器的原理框图

顶层模块的编写代码如下：

```
module ModeComparator(aIn,bIn,m,out);
    input [7:0] aIn,bIn;
    input m;
    output [7:0] out;

    //比较器示例
    wire agb;
    comp #(.n(8))comp_inst(.a(aIn), .b(bIn), .agb(agb), .aeb(), .alb());

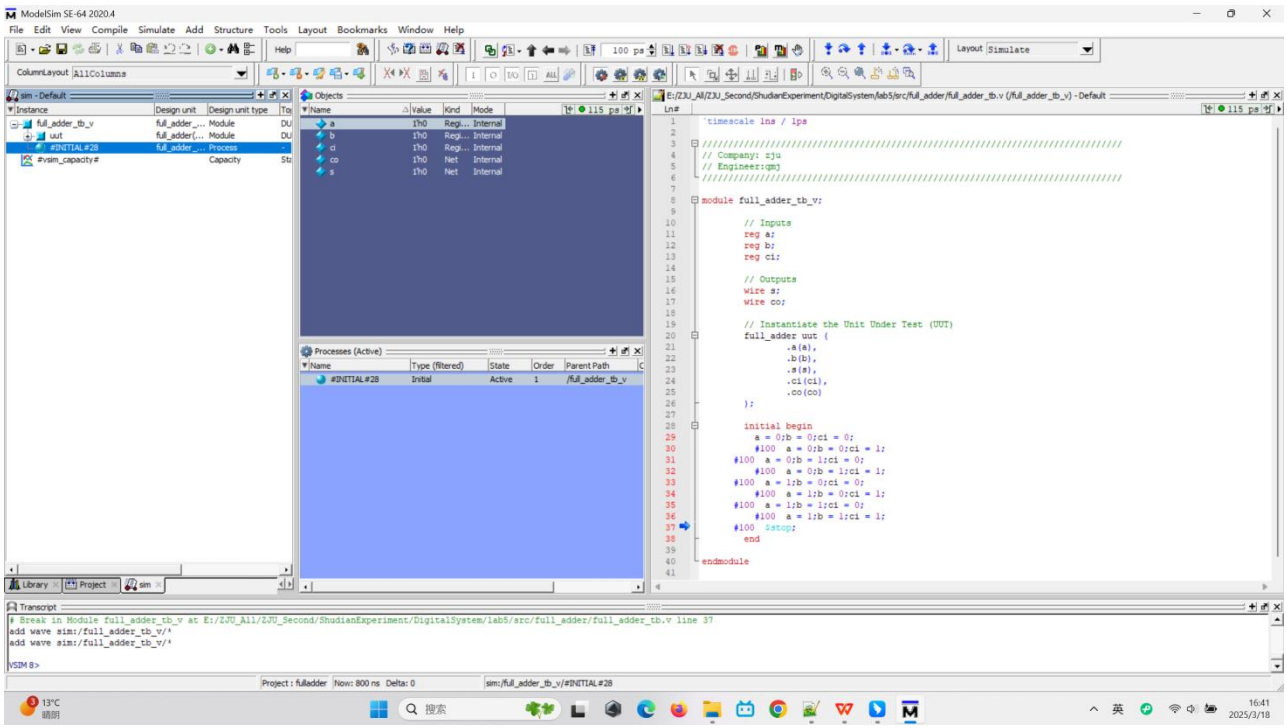
    //数据选择器
    mux_2to1 #(.n(8))mux(.out(out), .in0(aIn), .in1(bIn), .addr(~agb^m));
endmodule
```

（2）电路模块设计：与实验一相同

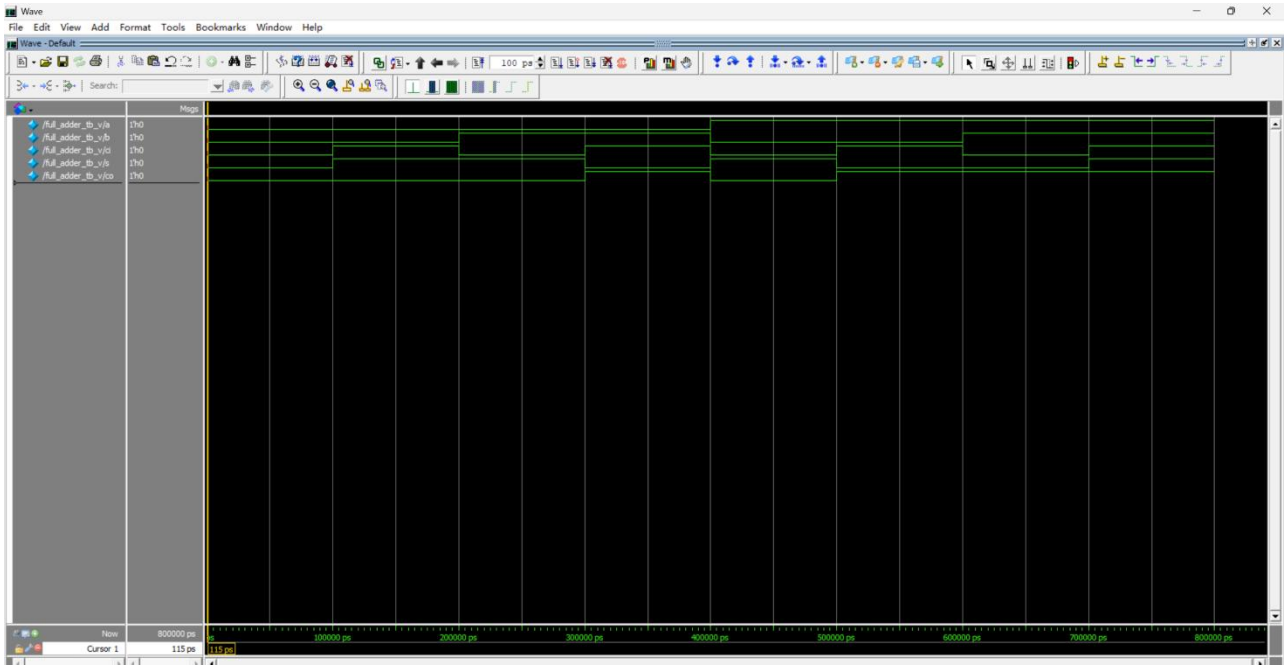
三、ModelSim 的仿真

1．电路模块全加器的仿真

下图为仿真界面，执行 tb 文件的 Simulate 后的情况



得到的 Wave 波形图



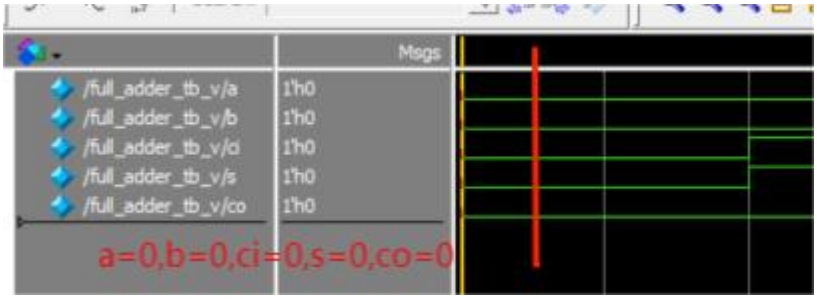
分析 Wave 波形图：由于全加器的逻辑函数为

$$\begin{cases} S_i = A_i \oplus B_i \oplus C_{i-1} \\ C_i = A_i B_i + B_i C_{i-1} + A_i C_{i-1} \end{cases}$$

阅读 tb 文件可以得到它的测试点

```
initial begin
  a = 0;b = 0;ci = 0;
  #100  a = 0;b = 0;ci = 1;
  #100  a = 0;b = 1;ci = 0;
  #100  a = 0;b = 1;ci = 1;
  #100  a = 1;b = 0;ci = 0;
  #100  a = 1;b = 0;ci = 1;
  #100  a = 1;b = 1;ci = 0;
  #100  a = 1;b = 1;ci = 1;
  #100  $stop;
end
```

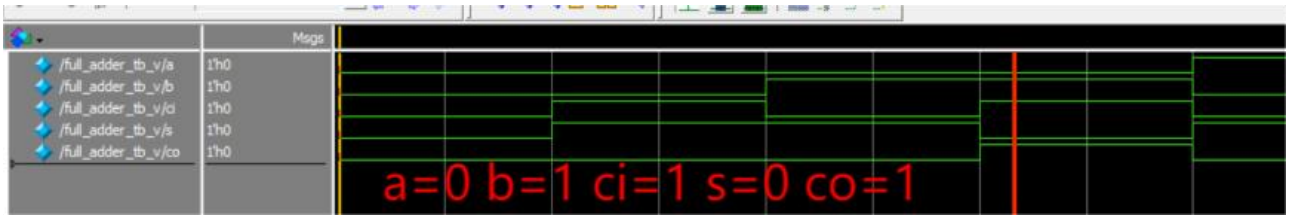
由波形图中 a，b，ci 均处于低电平时，s，co 均处于低电平；



a,b 处于低电平，ci 处于高电平时，s 处于高电平，co 处于低电平；



a 处于低电平，b，ci 处于高电平时，s 处于低电平，co 处于高电平；



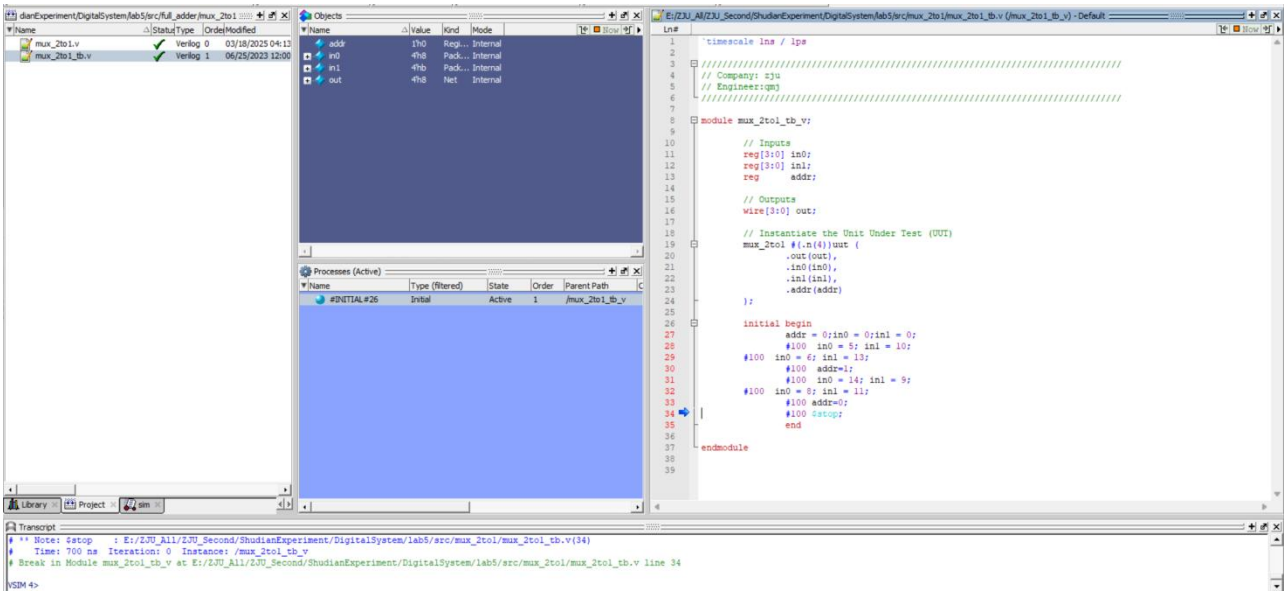
a，b，ci 均处于高电平时，s，co 均处于高电平，



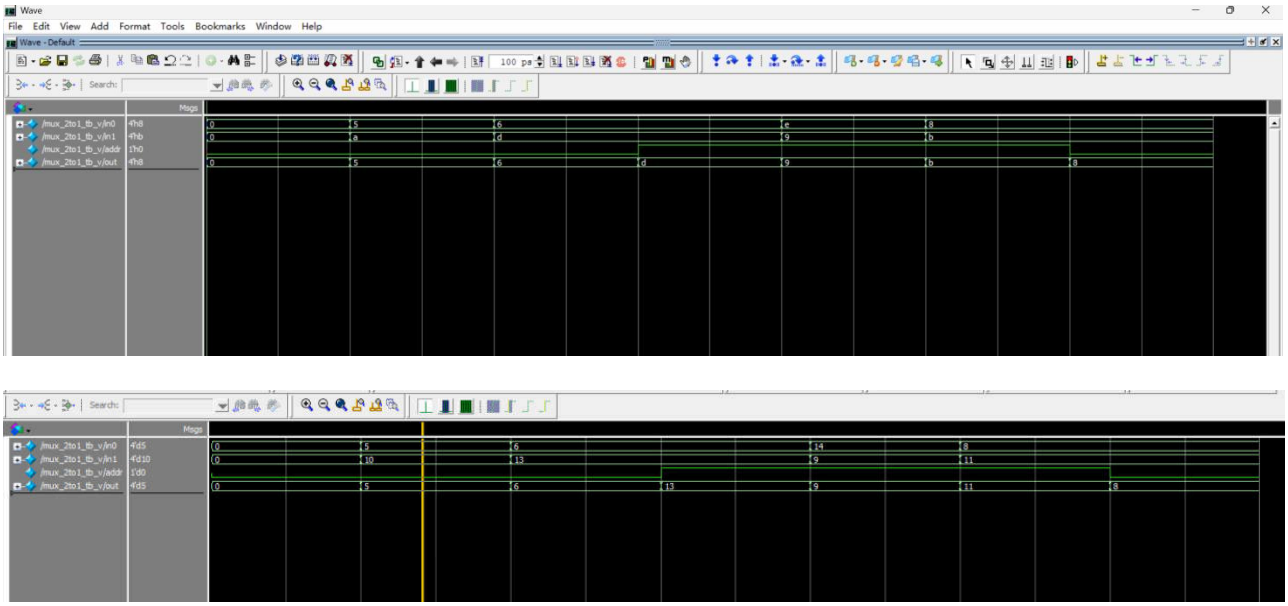
因此该代码可以实现 1bit 加法器的功能。

2．电路模块数据选择器的仿真

下图为仿真界面，执行 tb 文件的 Simulate 后的情况



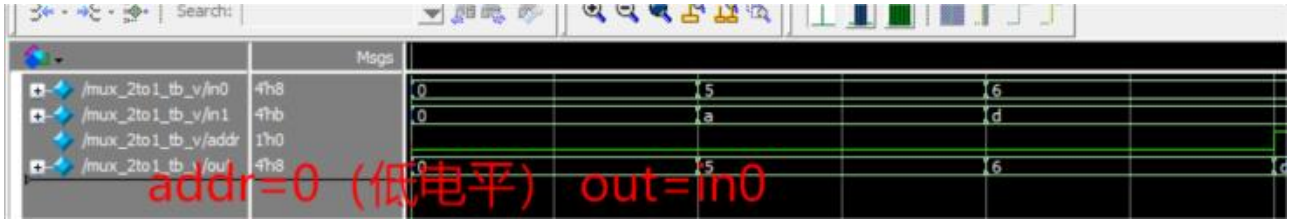
得到的 Wave 波形图（图 2 转化为十进制）

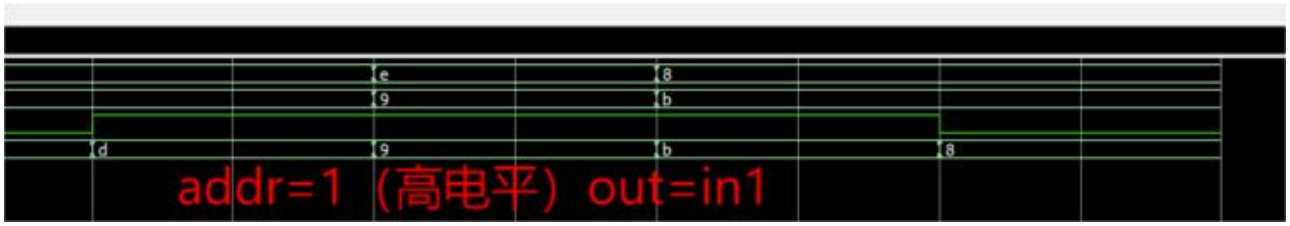


阅读 tb 文件可以得到它的测试点

```
initial begin
    addr = 0;in0 = 0;in1 = 0;
    #100  in0 = 5; in1 = 10;
    #100  in0 = 6; in1 = 13;
    #100  addr=1;
    #100  in0 = 14; in1 = 9;
    #100  in0 = 8; in1 = 11;
    #100 addr=0;
    #100 $stop;
    end
```

由波形图中 addr 为低电平时，out 的值和 in0 的值是相同的

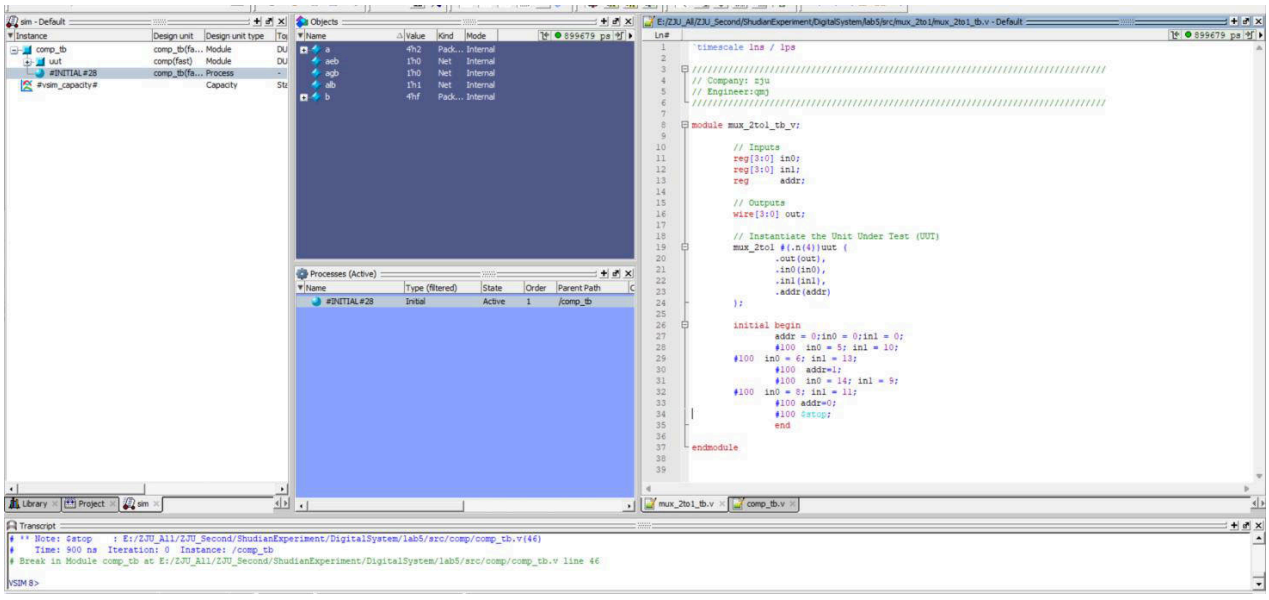




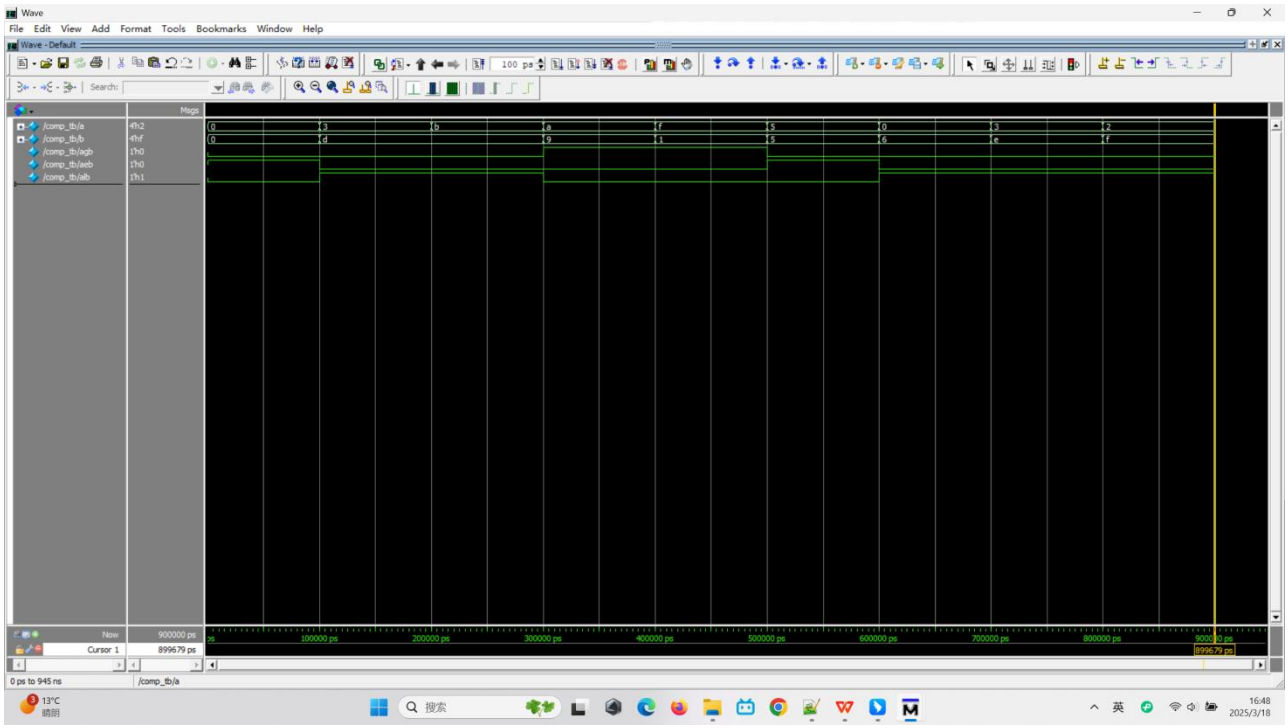
当 addr 为高电平时，out 的值和 in1 的值是相同的

3．电路模块数值比较器的仿真

下图为仿真界面，执行 tb 文件的 Simulate 后的情况



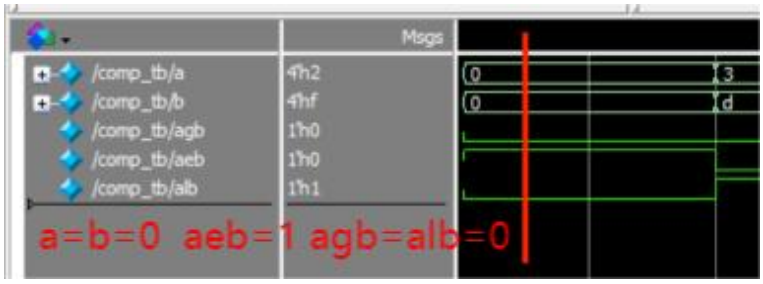
得到的 Wave 波形图



阅读 tb 文件可以得到它的测试点

```
initial begin
    a = 0;      b = 0;
    #100
    a = 4'b0011;    b = 4'b1101;
    #100
    a = 4'b1011;    b = 4'b1101;
    #100
    a = 4'b1010;    b = 4'b1001;
    #100
    a = 4'b1111;    b = 4'b0001;
    #100
    a = 4'b0101;    b = 4'b0101;
    #100
    a = 4'b0000;    b = 4'b0110;
    #100
    a = 4'b0011;    b = 4'b1110;
    #100
    a = 4'b0010;    b = 4'b1111;
    #100  $stop;
end
```

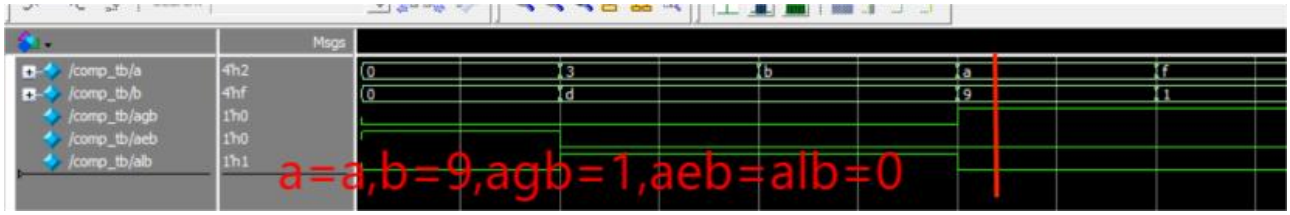
当 a 和 b 均为 0 时，agb、alb 为低电平，aeb 为高电平，说明 a 和 b 相等；



当 a=3 或 b，b=d 时，alb 为高电平，agb、aeb 为低电平，说明 a 比 b 小



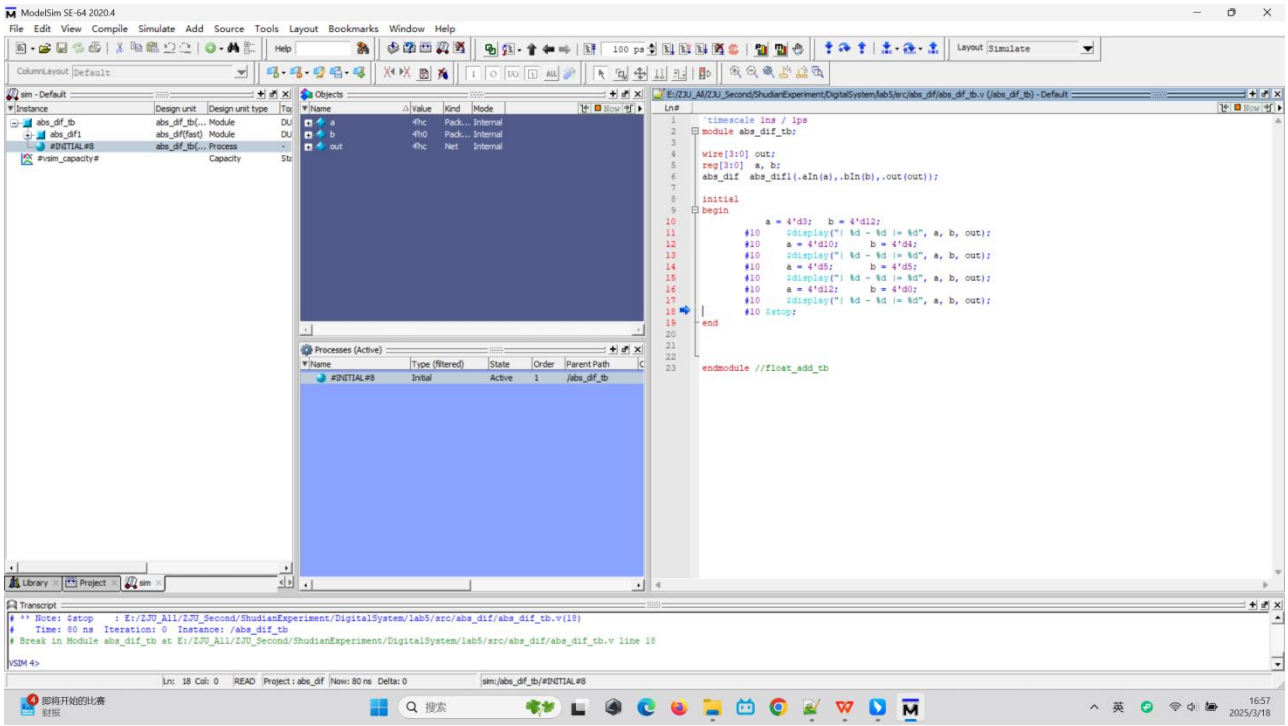
当 a=a（十六进制转十进制后为 10），b=9 时，agb 为高电平，aeb、alb 为低电平，说明 a 比 b 大；



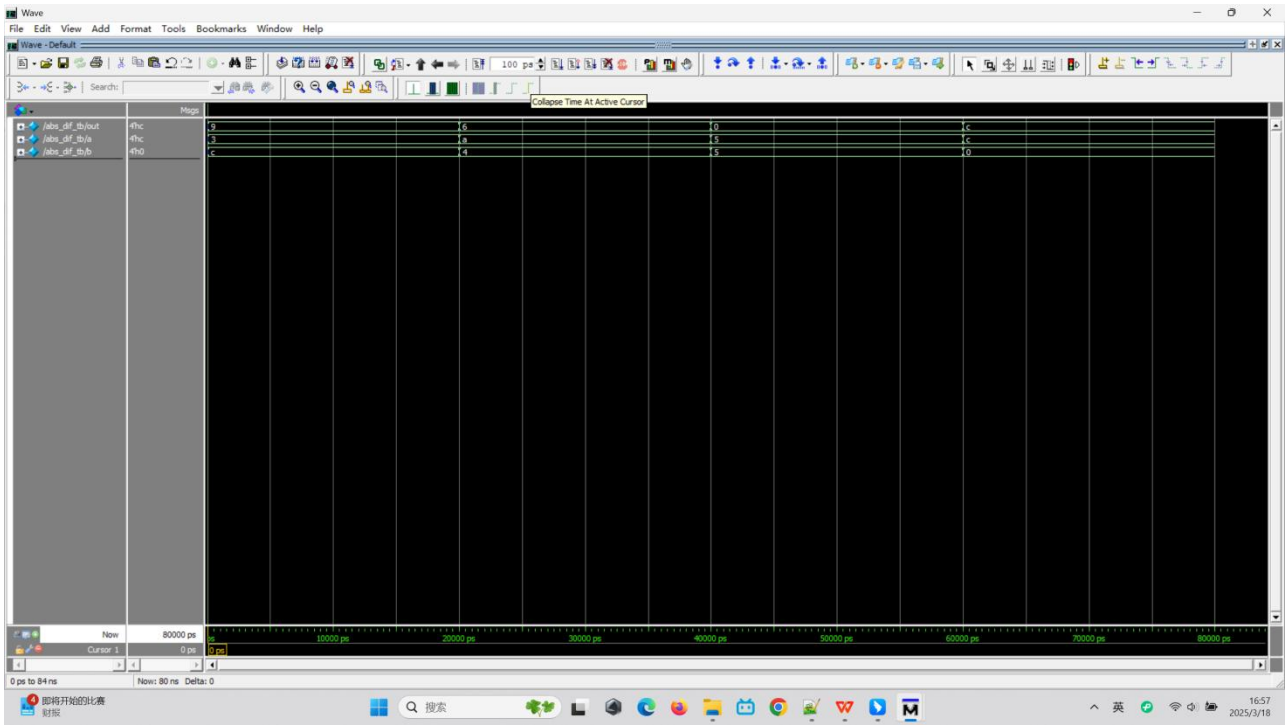
上述测试点证明了数值比较器的功能是正常的。

4．顶层模块绝对值电路 abs_dif 的仿真

下图为仿真界面，执行 tb 文件的 Simulate 后的情况



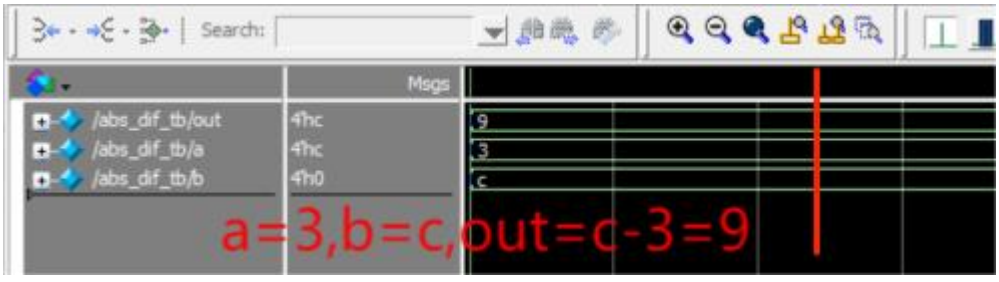
得到的 Wave 波形图



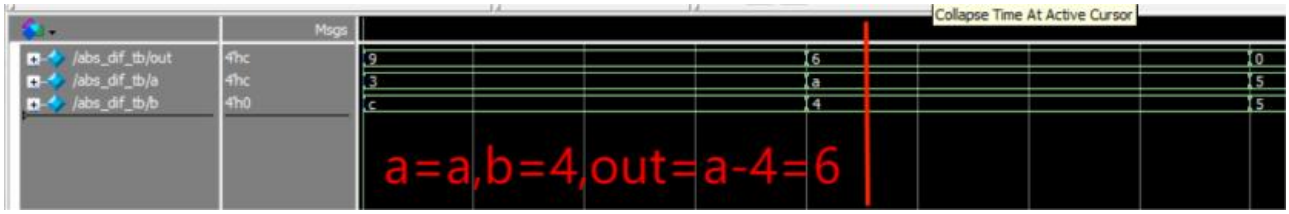
阅读 tb 文件可以得到它的测试点

```
begin
        a = 4'd3;    b = 4'd12;
    #10 $display("| %d - %d |= %d", a, b, out);
    #10 a = 4'd10;   b = 4'd4;
    #10 $display("| %d - %d |= %d", a, b, out);
    #10 a = 4'd5;    b = 4'd5;
    #10 $display("| %d - %d |= %d", a, b, out);
    #10 a = 4'd12;   b = 4'd0;
    #10 $display("| %d - %d |= %d", a, b, out);
    #10 $stop;
end
```

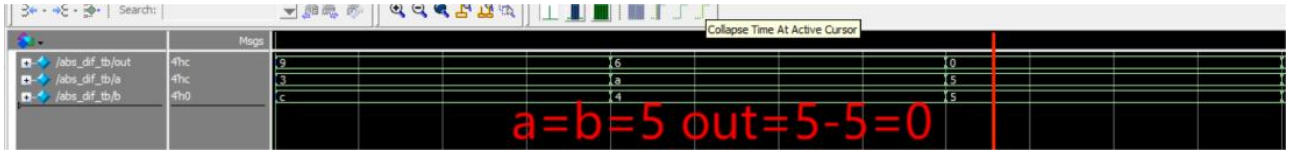
由波形图中，a=3，b=c（十六进制转为 12）时，out=9，符合绝对值之差；



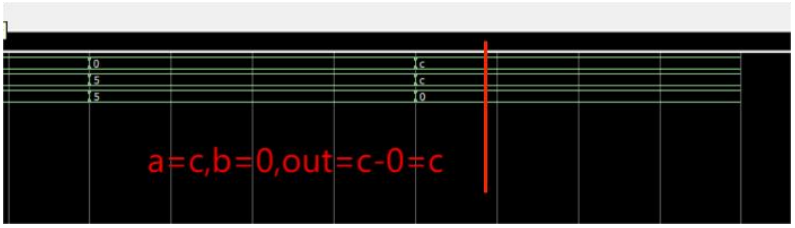
a=a（十六进制转为 10），b=6，out=4，符合；



a=5，b=5 时，out=0，符合；

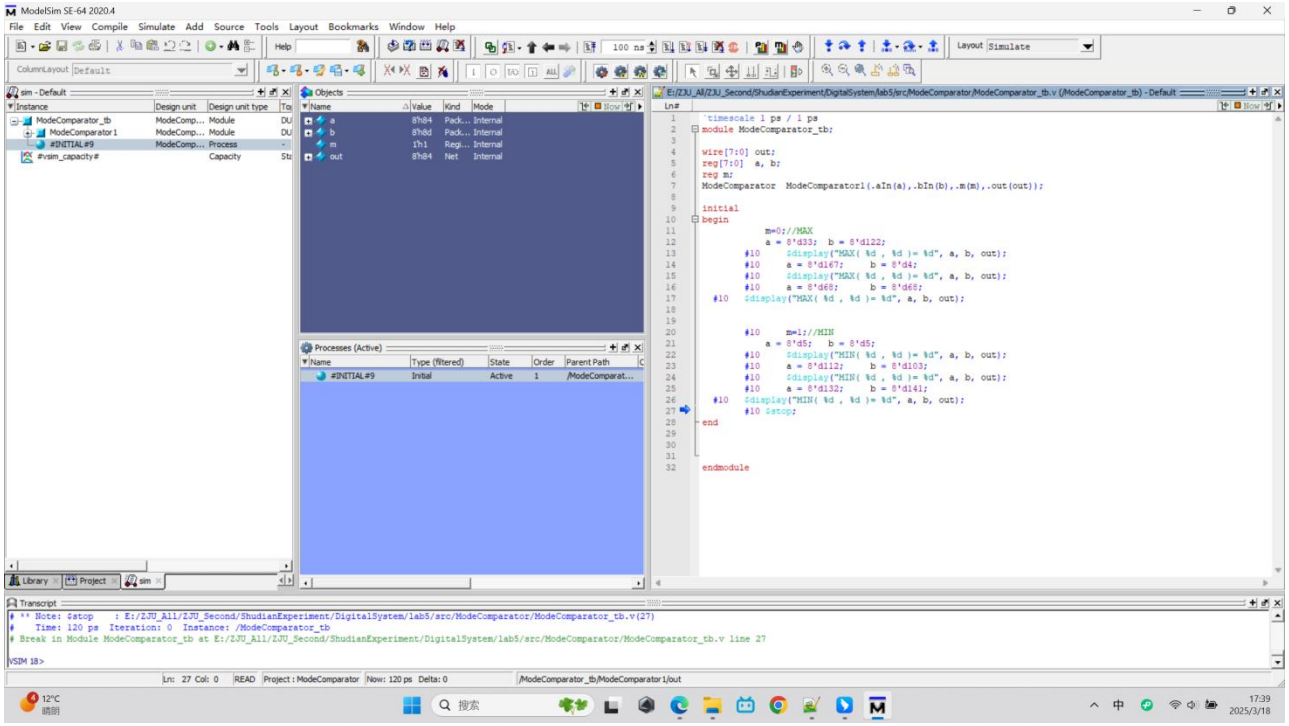


a=c，b=0 时，out=c，符合

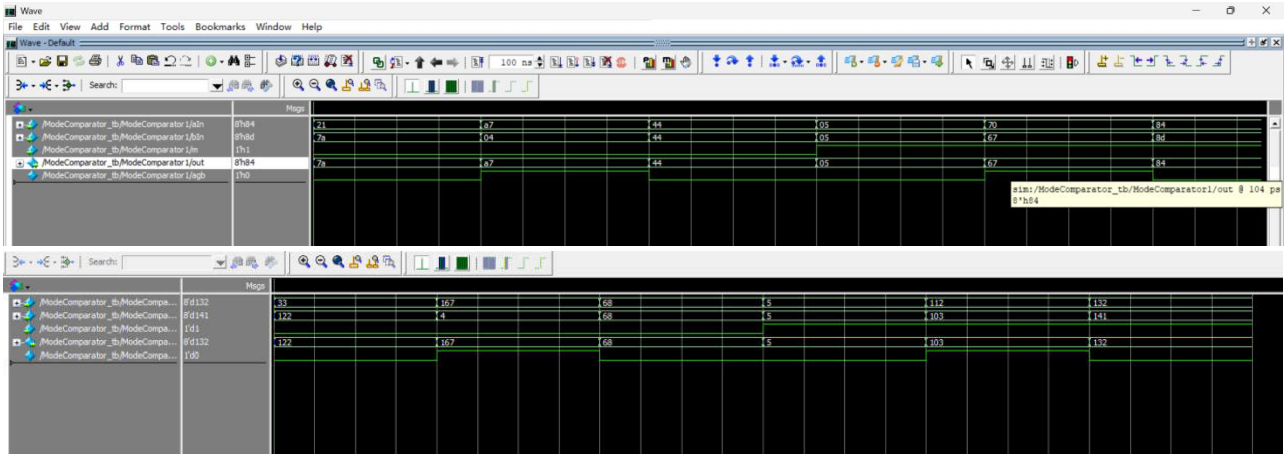


5．顶层模块模式比较器的仿真

下图为仿真界面，执行 tb 文件的 Simulate 后的情况



得到的 Wave 波形图（图 2 为转化为十进制）



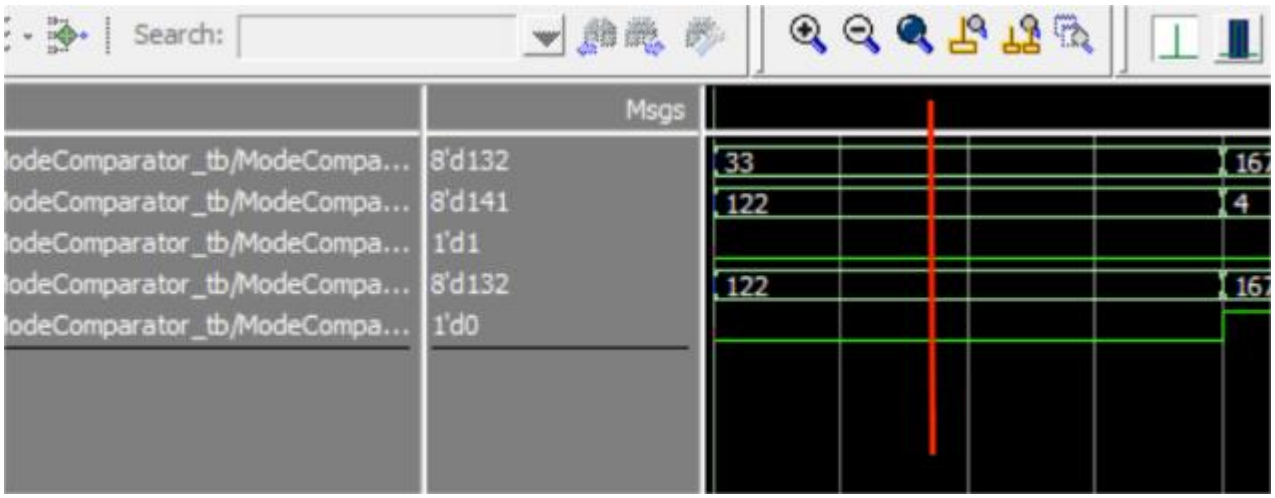
阅读 tb 文件可以得到它的测试点

```
begin
        m=0;//MAX
        a = 8'd33;  b = 8'd122;
    #10 $display("MAX( %d , %d )= %d", a, b, out);
    #10 a = 8'd167; b = 8'd4;
    #10 $display("MAX( %d , %d )= %d", a, b, out);
    #10 a = 8'd68;  b = 8'd68;
  #10   $display("MAX( %d , %d )= %d", a, b, out);

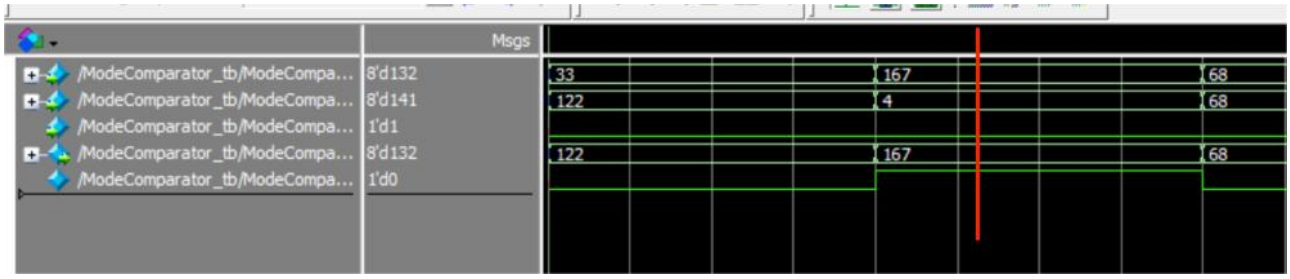

    #10 m=1;//MIN
        a = 8'd5;   b = 8'd5;
    #10 $display("MIN( %d , %d )= %d", a, b, out);
    #10 a = 8'd112; b = 8'd103;
    #10 $display("MIN( %d , %d )= %d", a, b, out);
    #10 a = 8'd132; b = 8'd141;
  #10   $display("MIN( %d , %d )= %d", a, b, out);
    #10 $stop;
end
```

由波形图中 当 m=0 低电平时，应该输出 MAX（a，b）

a=21（转为十进制为 33），b=7a（转为十进制 122）时，out=7a（转为十进制 122），agb=0，即 a 不大于 b，输出的确实最大值 b

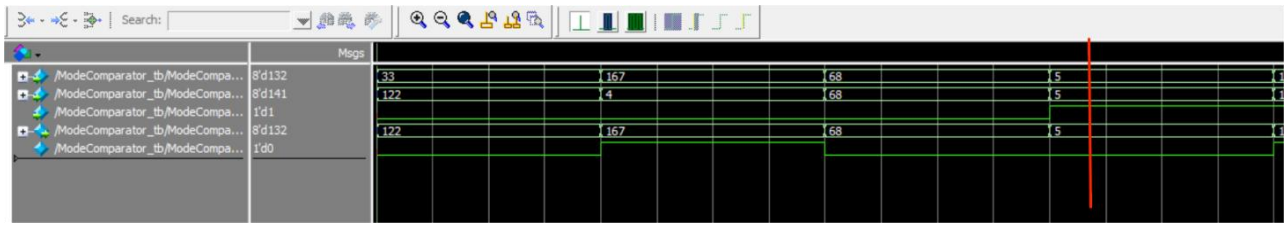


a=a7（转为十进制 167），b=04（转为十进制 4）时，out=a7（转为十进制 167），agb=1，即 a 大于 b，输出的也是最大值 a

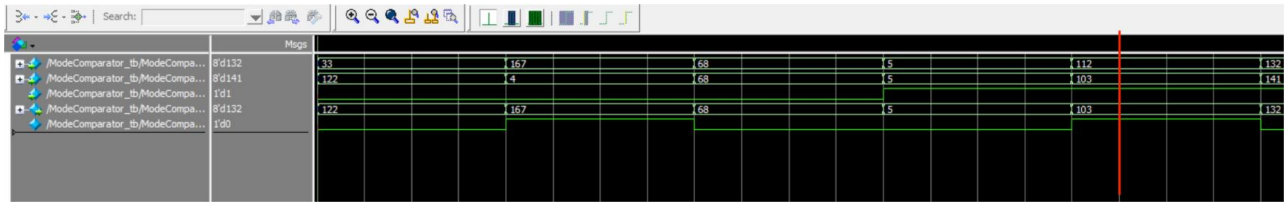


当 m=1 高电平时，应输出 MIN（a，b）

a=05（转为十进制 5），b=05（转为十进制 5），out=05（转为十进制 5）agb=0，即 a 不大于 b，输出的确实最小值 a 或 b（此时 a、b 正好相等）



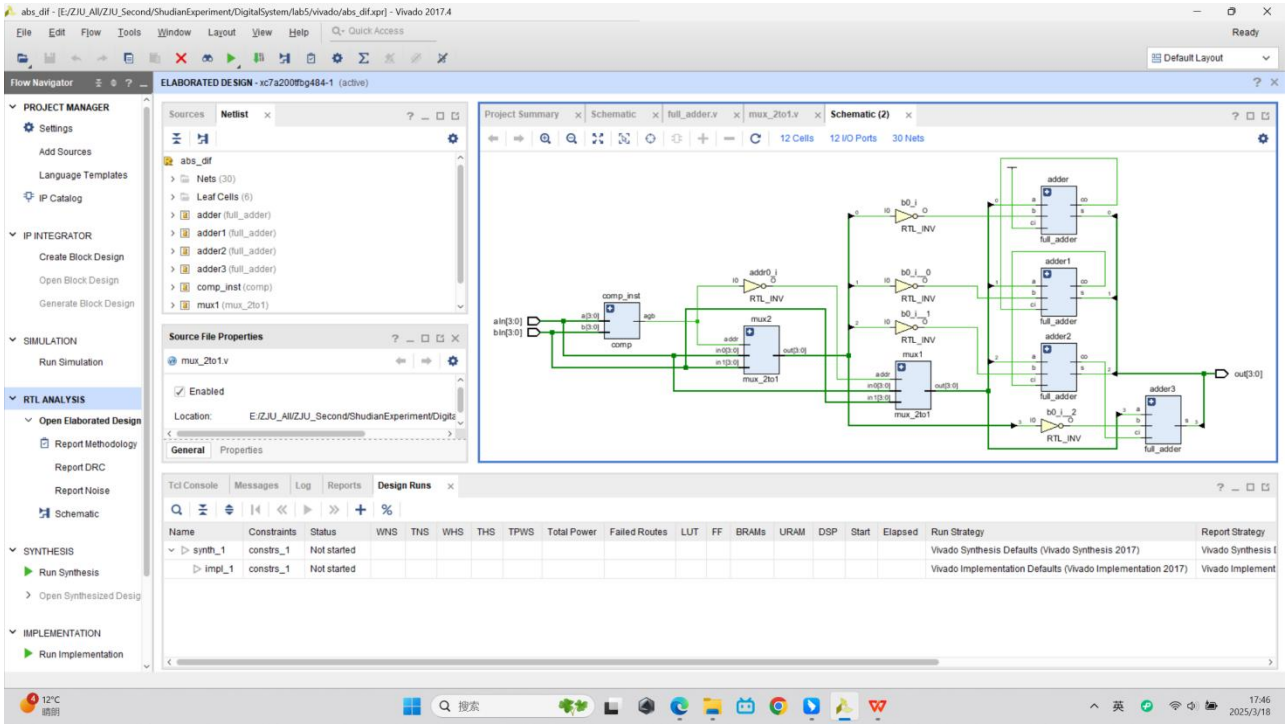
a=70（转为十进制 112），b=67（转为十进制 103），out=67（转为十进制 103），agb=1 为高电平，即 a 大于 b，输出的是最小值 b，符合功能要求



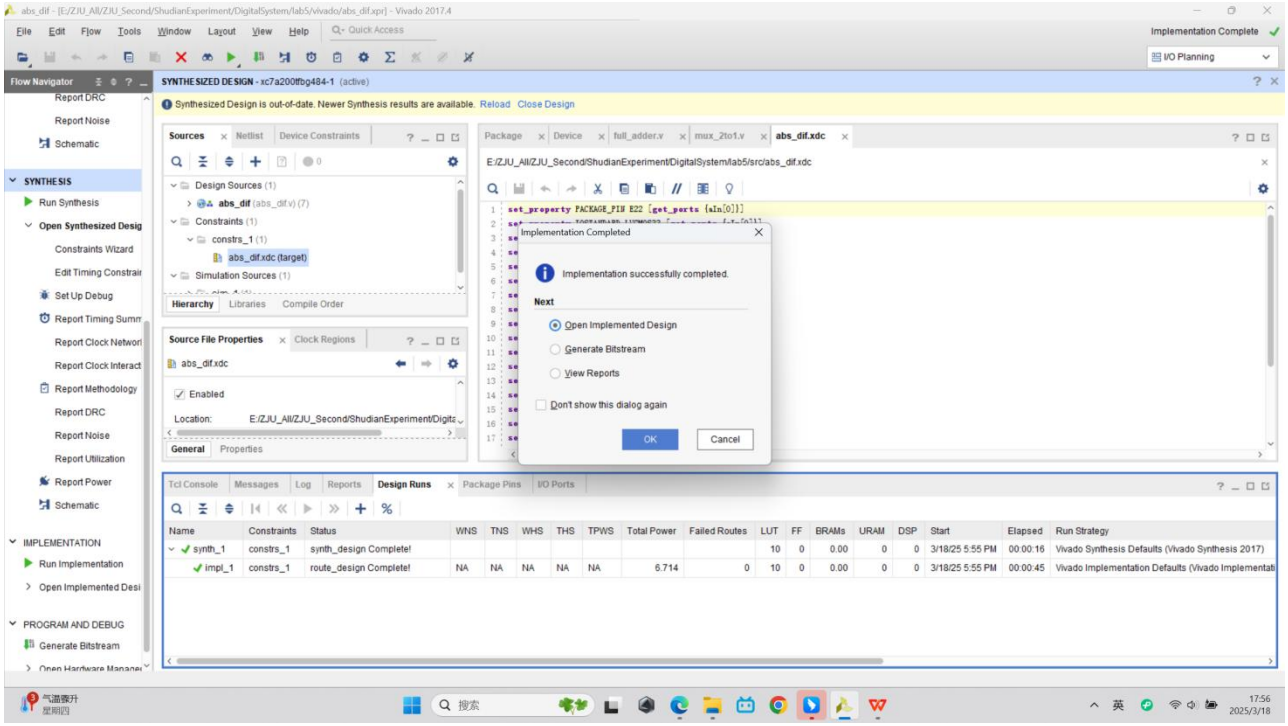
四、实验结果和开发板测试

1．vivado 工程文件建立

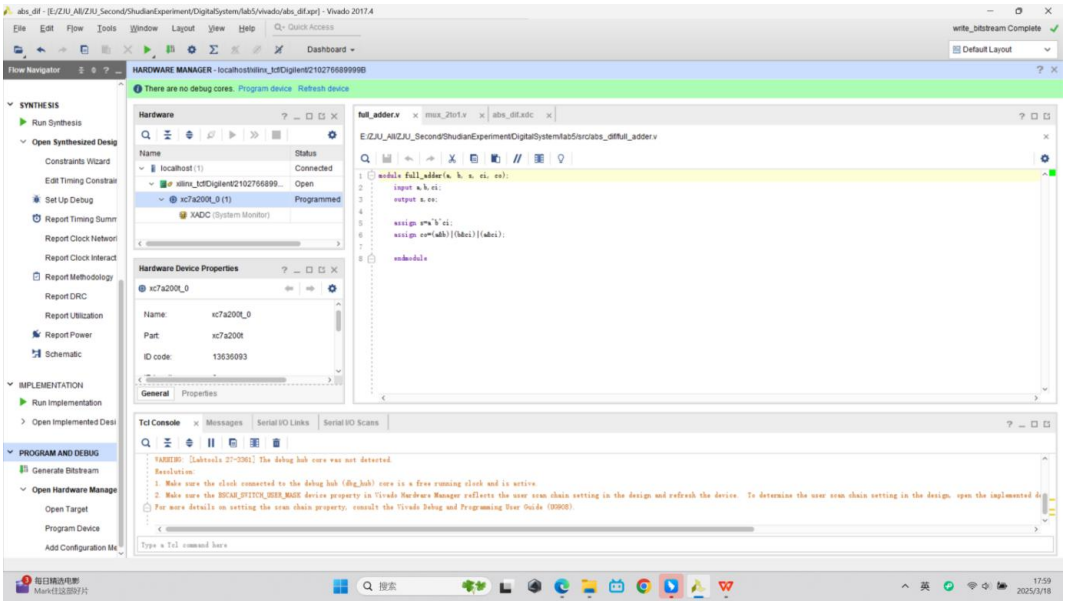
（1）RTL 综合



（2）引脚约束



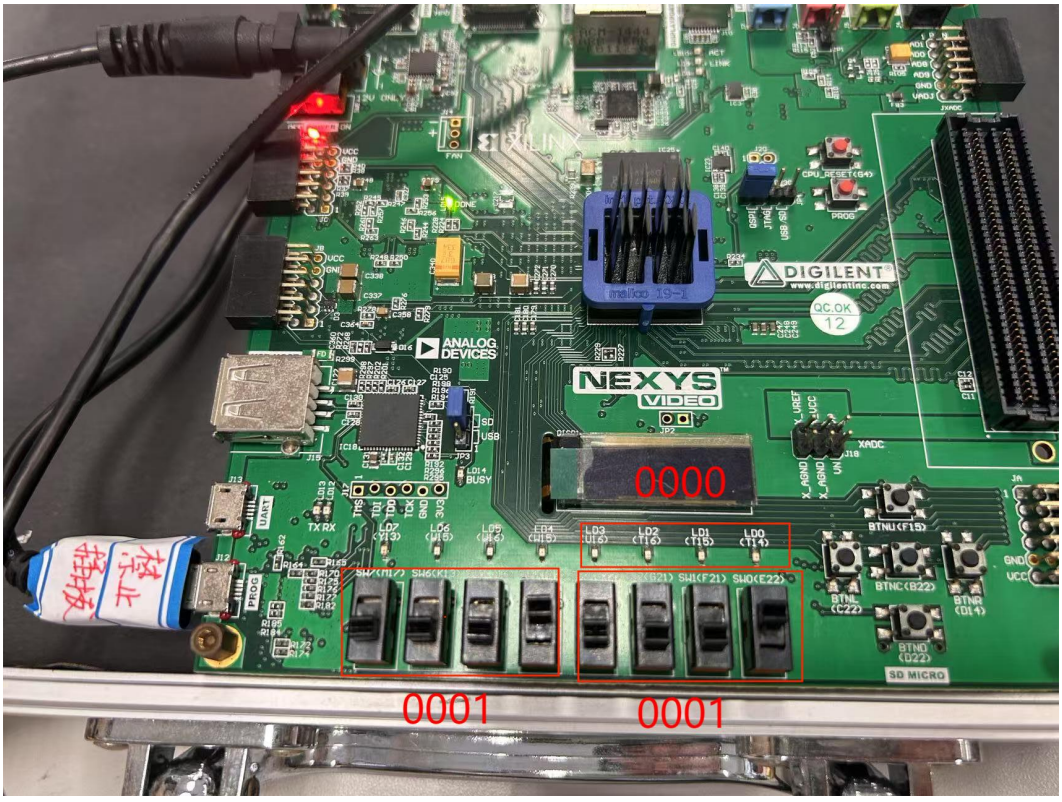
（3）连接开发板



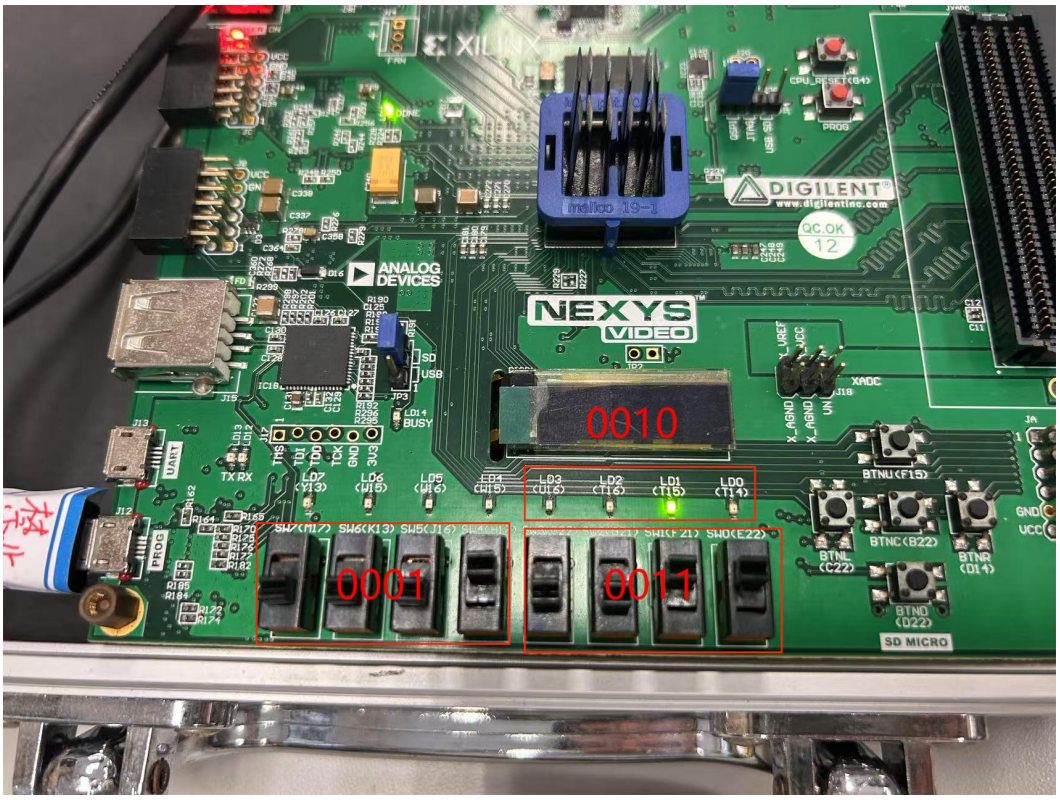
2．开发板测试：以下为 abs_dif 功能的板子测试

（1）引脚分析，SW0-SW3，四个开关表示 aIn 的四位输入，SW4-SW7，四个开关表示 bIn 的四位输入，LED0-LED3 表示最后的 out 情况

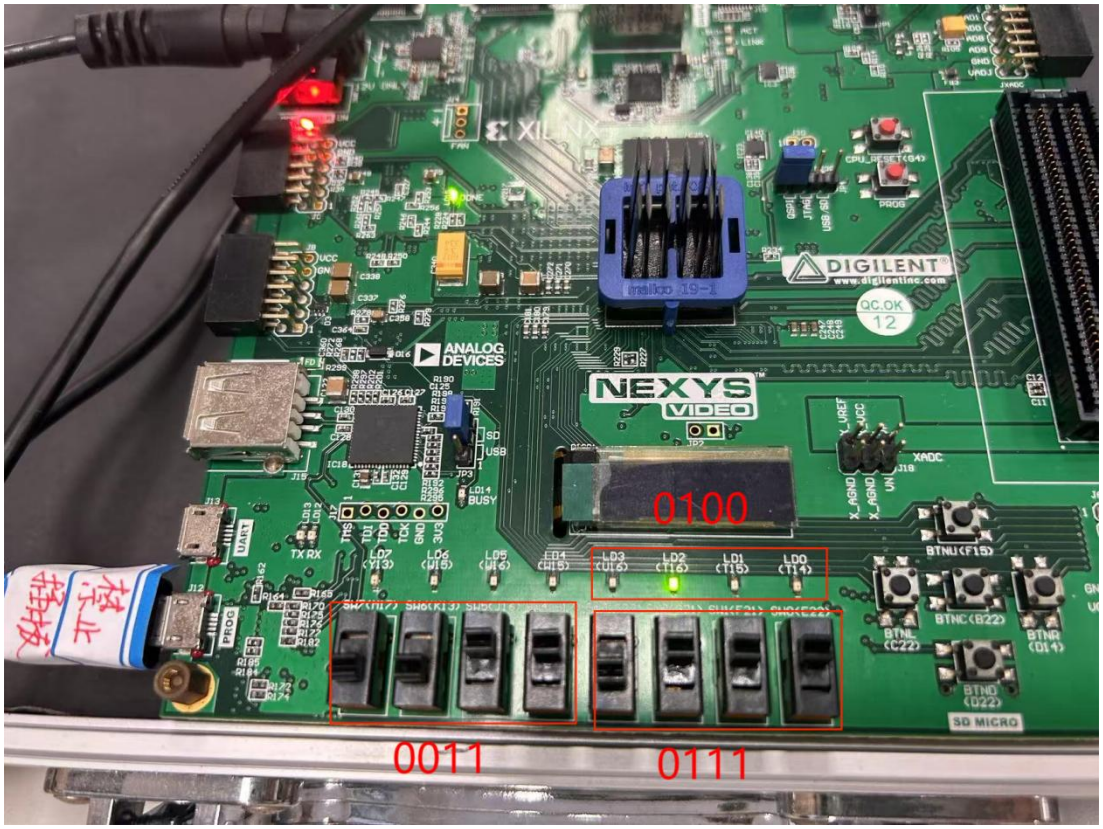
（2）测试 1：aIn=0001；bIn=0001 ；out=0000（LED0-3 均不亮）



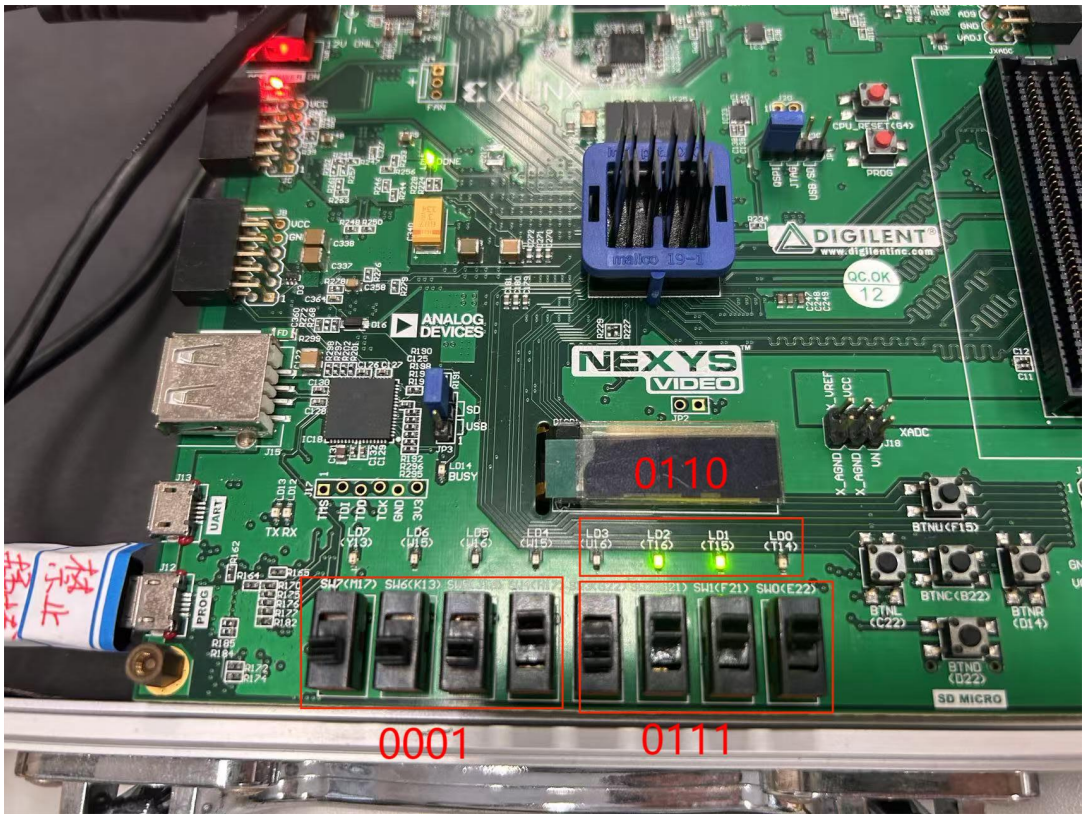
（3）测试 2：aIn=0011；bIn=0001；out=0010（仅 LED1 亮）



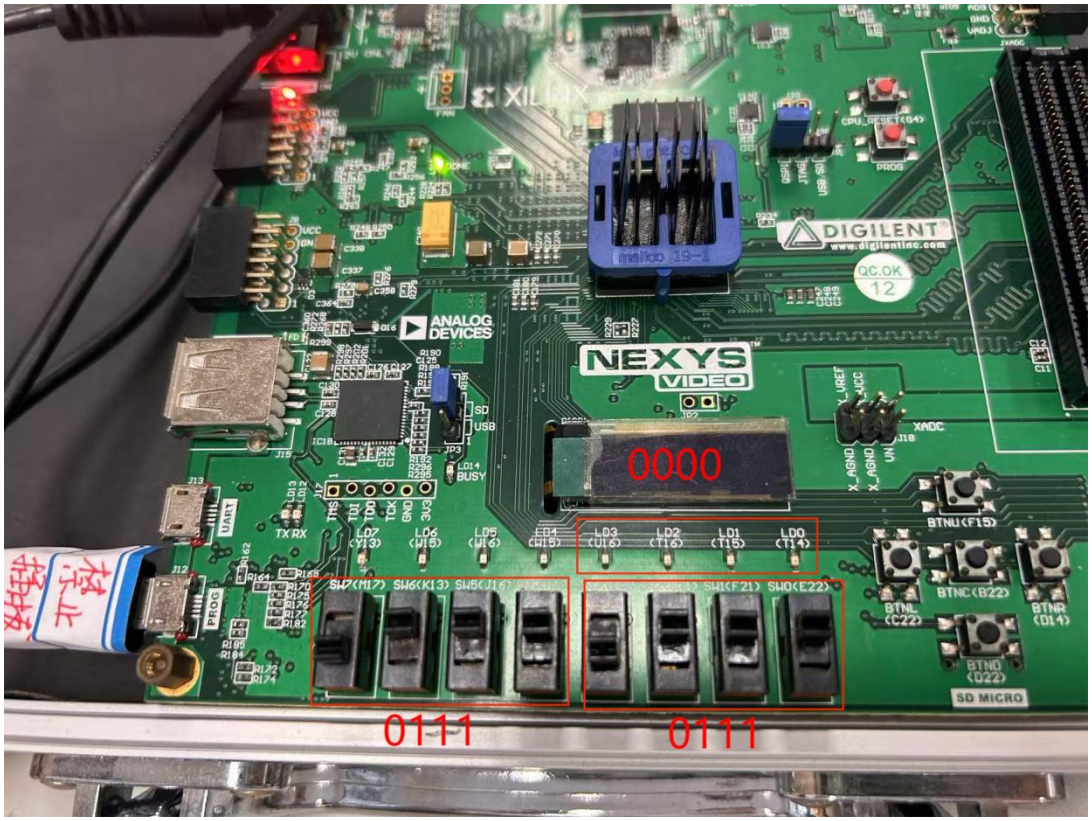
（4）测试 3：aIn=0111；bIn=0011；out=0100（仅 LED2 亮）



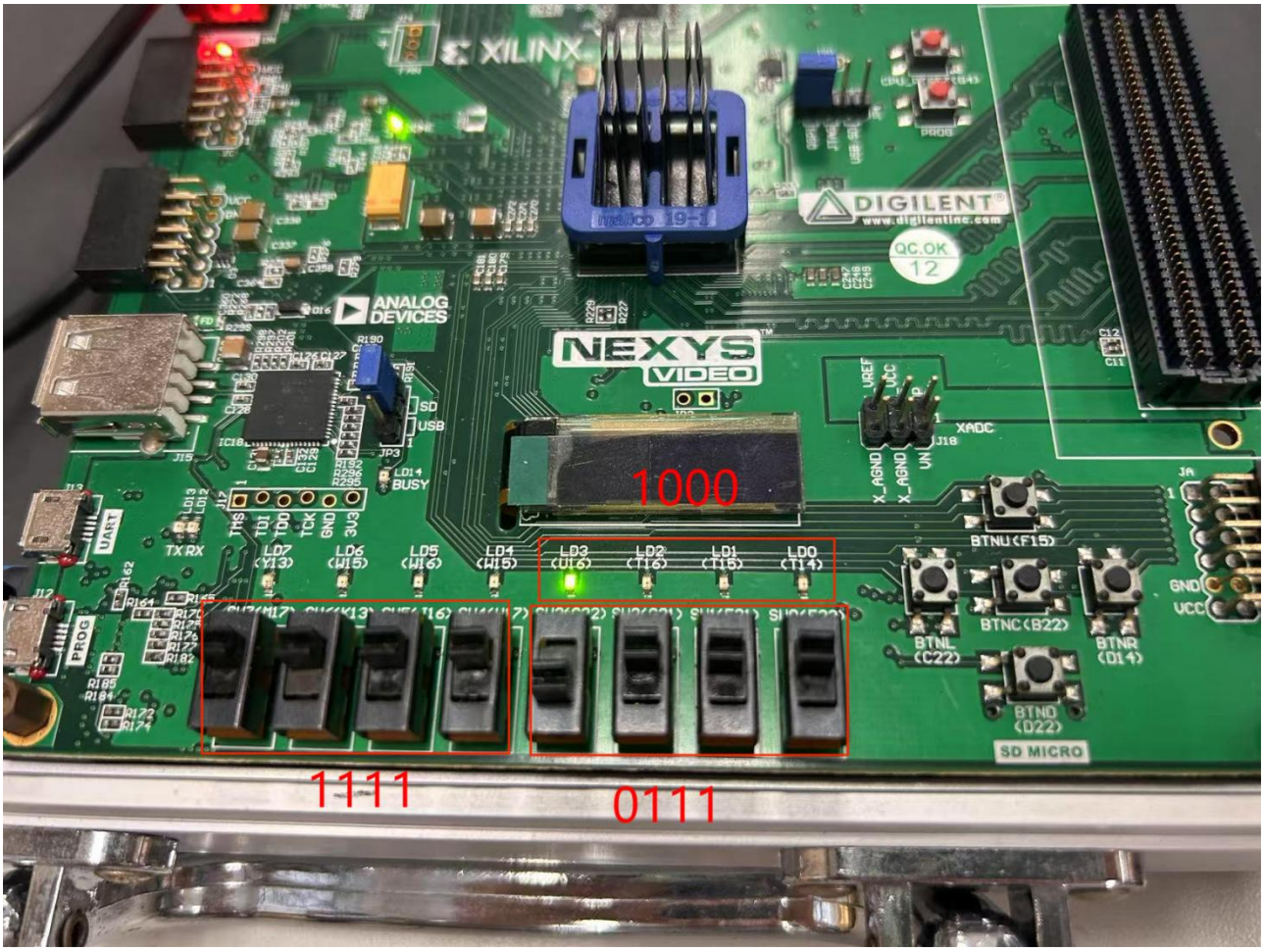
（5）测试 4：aIn=0111；bIn=0001；out=0110（LED1，LED2 亮）



（6）测试 5：aIn=0111；bIn=0111；out=0000（LED0--3 均不亮）



（7）测试 6：aIn=0111；bIn=1111；out=1000（LED3 亮）



五、实验过程中遇到的问题和解决方案

1．编写 verilog HDL 代码时：尤其是编写模块时，没有考虑使用 assign 语句和 always 过程语句，在 compile 时发生编译错误，assign 可以实现数值比较器和全加器的连续赋值（相当于一个循环 而不是一次性结束），always 可以实现过程性语句，数据选择器中 if-else 有判断的先后顺序，always 过程块中的代码是串行运行，有先后顺序，所以必须使用

2．使用 ModelSim 进行仿真时遇到的问题：

（1）一开始以为和 c 语言一样的调试过程，直接运行代码然后在命令运行窗口输入，但是学习过程中发现需要使用 tb（testbench）文件进行调试和验证，

然后观察 wave 图像去判断

（2）在编写 ModeComparator 顶层文件时，没有区分开端口和导线的区别，导致一开始 wave 图像没有输出曲线（是一条蓝线），观察代码才发现

```
module ModeComparator(aIn,bIn,m,out);
    input [7:0] aIn,bIn;
    input m;
    output [7:0] out;

    //比较器示例
    wire agb;
    comp #(.n(8))comp_inst(.a(aIn), .b(bIn), .agb(agb), .aeb(), .alb());

    //数据选择器
    mux_2to1 #(.n(8))mux(.out(out), .in0(aIn), .in1(bIn), .addr(~agb^m));
endmodule
```

蓝线是因为 out 没有连接在 mux 的 out 端口，而是中间引入了一个 wire [7:0] y 作为输出，导致有两个输出，但是 y 一开始并没有被声明为 output 所以导致最后没有结果可以输出来

3．使用 vivado 和开发板遇到的问题

六、思考题

1．尝试解决课本思考题

（1）求两数之差的绝对值电路，若输入为有符号数（补码），该怎么设计？

A：原设计为无符号的四 bit 的绝对值电路，当有符号时，可以借助符号位判断差值的正负，首先用反码实现 A-B=A+（~B+1），然后借助得到结果的符号位判断正负，若大于 0 则取原码，若小于 0 则取反码

（2）能否用加法器实现比较器？

A：可以借助求两数之差的想法，求解 A-B=A+（~B+1），然后判断符号位，如果符号位为 1，则 A<B；如果符号位为 0 且差不为 0，则 A>B；如果差为 0，A=B。

（3）调用模块时怎样进行参数的传递？若模块有多个参数，模块实例的格式如何？

A：1.可以从本题的例子看出来：

```
module  mux_2to1(out, in0,in1, addr);
parameter  n=1; //参数表示输入、输出数据的位数
output[n-1:0]  out;
input [n-1:0]   in0,in1;
input  addr;
```

```
comp #(.n(4))comp_inst(.a(aIn),  .b(bIn),  .agb(agb), .aeb(),  .alb());
//数据选择器实例，注意addr 信号的连接及参数的传递，注意实例名。
wire[3:0] max,min;
mux_2to1 #(.n(4))mux1(.out(max), .in0(aIn),  .in1(bIn), .addr(~agb));
mux_2to1 #(.n(4))mux2(.out(min), .in0(aIn),  .in1(bIn),  .addr(agb));
```

模块中需要先声明参数，然后调用模块时借助#（.n（））来为参数赋值

2.若模块中有多个参数，可以通过：

2.1 按参数顺序赋值

```
    parameter Width = 6, mem_num = 3;
    ......
endmodule

module Y(    //模块二，顶层。
    d,
    e,
    f
    );
    ......
    T #(5,7) T_inst0(   //这样的话Width=5, mem_num =7;
```

2.2 按参数顺序赋值

```
T #(.Width(5) ,
    .mem_num(7)
    )
```

2.3 利用 defparam 关键字

```
defparam T_inst0.Width = 5;
defparam T_inst0.mem_num = 7;
```

2．分析本实验以及总结

（1）本实验第一步需要了解电路如何搭建才可以实现想要的功能，就像数电课堂所学一样，可以列真值表，写逻辑函数，并且借助最基本的模块进行搭建，构建出原理图

（2）第二步就是利用 VerilogHDL 进行编写，可以先更具电路原理图，编写顶层代码，然后可以编写电路模块，如数值比较器，数据选择器，加法器等等，然后编译后利用 tb 文件检验代码是否正确，最后利用顶层 tb 文件检验整体代码是否正确，这一步需要利用 ModelSim 的仿真

（3）最后利用 Vivado 建立工程文件，观察 RTL 原理图，可以进一步检验代码的正确性，然后设置引脚约束和时序约束（本实验不需要），然后将电脑与板子相连，将程序上传，然后看板子能否实现相关功能